Academic Year: 2026-27
Semester: V
Class / Div / Branch: T.E./ C / CSE(AI&ML)
Subject: WMD Lab
Name of Instructor: Prof. Vijesh Nair

Name of Student: Neev Rambhia
Student ID: 25206010
Roll No: 154
Date of Performance: 26/08/2026
Date of Submission: 09/09/2026

Experiment No. 4

Aim: Implement State Management using Hooks and Context API; add form handling and validation.

Lab Outcomes: Develop responsive front-end applications using React components and state management.

Software Used: Javascript, Nodejs, React

Program:

App.jsx:

```
import React from 'react';

import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';


import AppShell from './components/layout/AppShell';

import { CampusFixProvider, useCampusFix } from './context/CampusFixContext';


// Pages

import Dashboard from './pages/Dashboard';

import Issues from './pages/Issues';

import IssueDetails from './pages/IssueDetails';

import ReportIssue from './pages/ReportIssue';

import MyReports from './pages/MyReports';

import Announcements from './pages/Announcements';
```

```
import Help from './pages/Help';

import Analytics from './pages/Analytics';

import MaintenanceTeam from './pages/MaintenanceTeam';


import './index.css';


function AppContent() {
 const {
  dataSource,
  setDataSource,
  loadIssues,
  searchTerm,
  setSearchTerm
 } = useCampusFix();


 const handleToggleDataSource = () => {
  if (dataSource === 'api') {
   setDataSource('mock');
  } else {
   loadIssues();
  }
 };


 return (
  <Router>
```

```
<AppShell
  dataSource={dataSource}
  onToggleDataSource={handleToggleDataSource}
  searchTerm={searchTerm}
  onSearchChange={(e) => setSearchTerm(e.target.value)}
>
  <Routes>
    {/* Default Redirection */}
    <Route path="/" element={<Navigate to="/dashboard" replace />} />

    {/* Page 1: Dashboard Overview */}
    <Route path="/dashboard" element={<Dashboard />} />

    {/* Page 2: Issues Management */}
    <Route path="/issues" element={<Issues />} />

    {/* Page 2 Detail: Issue Details */}
    <Route path="/issues/:id" element={<IssueDetails />} />

    {/* Page 3: Report Issue */}
    <Route path="/report" element={<ReportIssue />} />

    {/* Page 4: My Reports */}
    <Route path="/my-reports" element={<MyReports />} />
```

```
        {/* Page 5: Announcements */}

        <Route path="/announcements" element={<Announcements />} />


        {/* Page 6: Help & Support */}

        <Route path="/help" element={<Help />} />


        {/* Page 7: Analytics */}

        <Route path="/analytics" element={<Analytics />} />


        {/* Optional Roster Route */}

        <Route path="/maintenance-team" element={<MaintenanceTeam />} />


        {/* Fallback 404 */}

        <Route path="*" element={<Navigate to="/dashboard" replace />} />

      </Routes>

    </AppShell>

  </Router>

 );

}


export function App() {

 return (

  <CampusFixProvider>

   <AppContent />

  </CampusFixProvider>
```

```
  );
}


export default App;
```

App.css:

```
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
```

```
position: relative;


.base,
.framework,
.vite {
  inset-inline: 0;
  margin: 0 auto;
}


.base {
  width: 170px;
  position: relative;
  z-index: 0;
}


.framework,
.vite {
  position: absolute;
}


.framework {
  z-index: 1;
  top: 34px;
  height: 28px;
  transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
```

```
    scale(1.4);
  }

  .vite {
    z-index: 0;
    top: 107px;
    height: 26px;
    width: auto;
    transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
      scale(0.8);
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
```

```
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
  text-align: left;

  & > div {
    flex: 1 1 0;
    padding: 32px;
    @media (max-width: 1024px) {
      padding: 24px 20px;
    }
  }

  .icon {
    margin-bottom: 16px;
    width: 22px;
    height: 22px;
  }

  @media (max-width: 1024px) {
    flex-direction: column;
    text-align: center;
  }
```

```
}

#docs {
  border-right: 1px solid var(--border);

  @media (max-width: 1024px) {
    border-right: none;
    border-bottom: 1px solid var(--border);
  }
}

#next-steps ul {
  list-style: none;
  padding: 0;
  display: flex;
  gap: 8px;
  margin: 32px 0 0;

  .logo {
    height: 18px;
  }

  a {
    color: var(--text-h);
    font-size: 16px;
```

```
  border-radius: 6px;
  background: var(--social-bg);
  display: flex;
  padding: 6px 12px;
  align-items: center;
  gap: 8px;
  text-decoration: none;
  transition: box-shadow 0.3s;

  &:hover {
    box-shadow: var(--shadow);
  }
  .button-icon {
    height: 18px;
    width: 18px;
  }
}

@media (max-width: 1024px) {
  margin-top: 20px;
  flex-wrap: wrap;
  justify-content: center;

  li {
    flex: 1 1 calc(50% - 8px);
```

```
  }

  a {
    width: 100%;
    justify-content: center;
    box-sizing: border-box;
  }
 }
}

#spacer {
 height: 88px;
 border-top: 1px solid var(--border);
 @media (max-width: 1024px) {
  height: 48px;
 }
}

.ticks {
 position: relative;
 width: 100%;

 &::before,
 &::after {
  content: '';
```

```
  position: absolute;
  top: -4.5px;
  border: 5px solid transparent;
 }

 &::before {
  left: 0;
  border-left-color: var(--border);
 }
 &::after {
  right: 0;
  border-right-color: var(--border);
 }
}
```

CampusFixContext.jsx:

```
import React, { createContext, useContext, useState, useEffect, useMemo } from 'react';
import { user as initialUser, mockIssues } from '../data/mockData';
import {
 getIssues,
 getIssue,
 createIssue,
 updateIssue,
 patchIssue,
 deleteIssue
```

```
} from '../services/issueService';


// 1. Create Context

export const CampusFixContext = createContext(null);


// 2. Provider Component

export function CampusFixProvider({ children }) {

  // Application Data & Auth State

  const [issues, setIssues] = useState([]);

  const [currentUser, setCurrentUser] = useState(initialUser);

  const [selectedIssue, setSelectedIssue] = useState(null);


  // Search & Filter State

  const [searchTerm, setSearchTerm] = useState('');

  const [selectedCategory, setSelectedCategory] = useState('All');

  const [selectedPriority, setSelectedPriority] = useState('All');

  const [selectedStatus, setSelectedStatus] = useState('All');


  // Status & Connectivity State

  const [loading, setLoading] = useState(true);

  const [error, setError] = useState(null);

  const [dataSource, setDataSource] = useState('api'); // 'api' or 'mock'


  // Load Issues Action (GET /issues)

  const loadIssues = async () => {
```

```
  setLoading(true);
  setError(null);
  try {
    const data = await getIssues();
    setIssues(data);
    setDataSource('api');
  } catch (err) {
    console.warn('REST API unavailable. Falling back to mockData.js:', err.message);
    setError('Unable to connect to CampusFix API. Showing cached mock data.');
    setIssues(mockIssues);
    setDataSource('mock');
  } finally {
    setLoading(false);
  }
};

// Initial Load with useEffect
useEffect(() => {
  loadIssues();
}, []);

// Action: Add Issue (POST /issues)
const addIssue = async (newIssueData) => {
  setError(null);
  try {
```

```
    if (dataSource === 'api') {
      const created = await createIssue(newIssueData);
      setIssues((prev) => [created, ...prev]);
      return created;
    } else {
      const fallback = { ...newIssueData, id: String(Date.now()) };
      setIssues((prev) => [fallback, ...prev]);
      return fallback;
    }
  } catch (err) {
    console.error('API create issue failed:', err);
    setError(`Failed to create issue on server: ${err.message}`);
    const fallback = { ...newIssueData, id: String(Date.now()) };
    setIssues((prev) => [fallback, ...prev]);
    return fallback;
  }
};

// Action: Update Issue (PUT /issues/:id)
const handleUpdateIssue = async (id, updatedIssueData) => {
  try {
    if (dataSource === 'api') {
      const updated = await updateIssue(id, updatedIssueData);
      setIssues((prev) => prev.map((item) => (String(item.id) === String(id) ? updated : item)));
      if (selectedIssue && String(selectedIssue.id) === String(id)) {
```

```
      setSelectedIssue(updated);
    }
   } else {
    setIssues((prev) =>
     prev.map((item) => (String(item.id) === String(id) ? { ...item, ...updatedIssueData } : item))
    );
   }
  } catch (err) {
   console.error('API update failed:', err);
   setError(`Failed to replace issue: ${err.message}`);
  }
};

// Action: Patch Issue (PATCH /issues/:id)
const handlePatchIssue = async (id, patchData) => {
 try {
  if (dataSource === 'api') {
   const patched = await patchIssue(id, patchData);
   setIssues((prev) =>
    prev.map((item) => (String(item.id) === String(id) ? { ...item, ...patched } : item))
   );
   if (selectedIssue && String(selectedIssue.id) === String(id)) {
    setSelectedIssue((prev) => ({ ...prev, ...patched }));
   }
  } else {
```

```
    setIssues((prev) =>
      prev.map((item) => (String(item.id) === String(id) ? { ...item, ...patchData } : item))
    );
   }
  } catch (err) {
   console.error('API patch status failed:', err);
   setIssues((prev) =>
    prev.map((item) => (String(item.id) === String(id) ? { ...item, ...patchData } : item))
   );
  }
};

// Action: Delete Issue (DELETE /issues/:id)
const handleDeleteIssue = async (id) => {
 try {
  if (dataSource === 'api') {
   await deleteIssue(id);
  }
  setIssues((prev) => prev.filter((item) => String(item.id) !== String(id)));
  if (selectedIssue && String(selectedIssue.id) === String(id)) {
   setSelectedIssue(null);
  }
 } catch (err) {
  console.error('API delete failed:', err);
  setIssues((prev) => prev.filter((item) => String(item.id) !== String(id)));
```

```
  }
};


// Fetch single issue details
const fetchSingleIssue = async (id) => {
  try {
    const issue = await getIssue(id);
    setSelectedIssue(issue);
    return issue;
  } catch (err) {
    console.warn(`Could not fetch issue #${id} from API, finding in local state:`, err.message);
    const found = issues.find((i) => String(i.id) === String(id));
    if (found) setSelectedIssue(found);
    return found;
  }
};


// Derived Filtered Issues (non-mutating)
const filteredIssues = useMemo(() => {
  return issues.filter((issue) => {
    const titleText = (issue.title || '').toLowerCase();
    const descText = (issue.description || '').toLowerCase();
    const locText = (issue.location || '').toLowerCase();
    const reporterText = (issue.reporter || issue.reportedBy || '').toLowerCase();
    const query = searchTerm.toLowerCase();
```

```
    const matchesSearch =
      !searchTerm ||
      titleText.includes(query) ||
      descText.includes(query) ||
      locText.includes(query) ||
      reporterText.includes(query);

    const matchesCategory =
      selectedCategory === 'All' || issue.category === selectedCategory;

    const matchesPriority =
      selectedPriority === 'All' || issue.priority === selectedPriority;

    const matchesStatus =
      selectedStatus === 'All' || issue.status === selectedStatus;

    return matchesSearch && matchesCategory && matchesPriority && matchesStatus;
  });
}, [issues, searchTerm, selectedCategory, selectedPriority, selectedStatus]);

// Dynamic Dashboard Statistics derived from issues state
const stats = useMemo(() => {
  const totalIssues = issues.length;
  const reportedIssues = issues.filter((i) => i.status === 'Reported').length;
```

```
  const inProgressIssues = issues.filter((i) => i.status === 'In Progress').length;

  const resolvedIssues = issues.filter((i) => i.status === 'Resolved').length;

  const highPriorityIssues = issues.filter((i) => i.priority === 'High').length;


  return {
    totalIssues,
    reportedIssues,
    inProgressIssues,
    resolvedIssues,
    highPriorityIssues,
  };
}, [issues]);


const value = {
  // State Values
  issues,
  filteredIssues,
  currentUser,
  setCurrentUser,
  selectedIssue,
  setSelectedIssue,
  searchTerm,
  selectedCategory,
  selectedPriority,
  selectedStatus,
```

```
    loading,
    error,
    dataSource,
    stats,

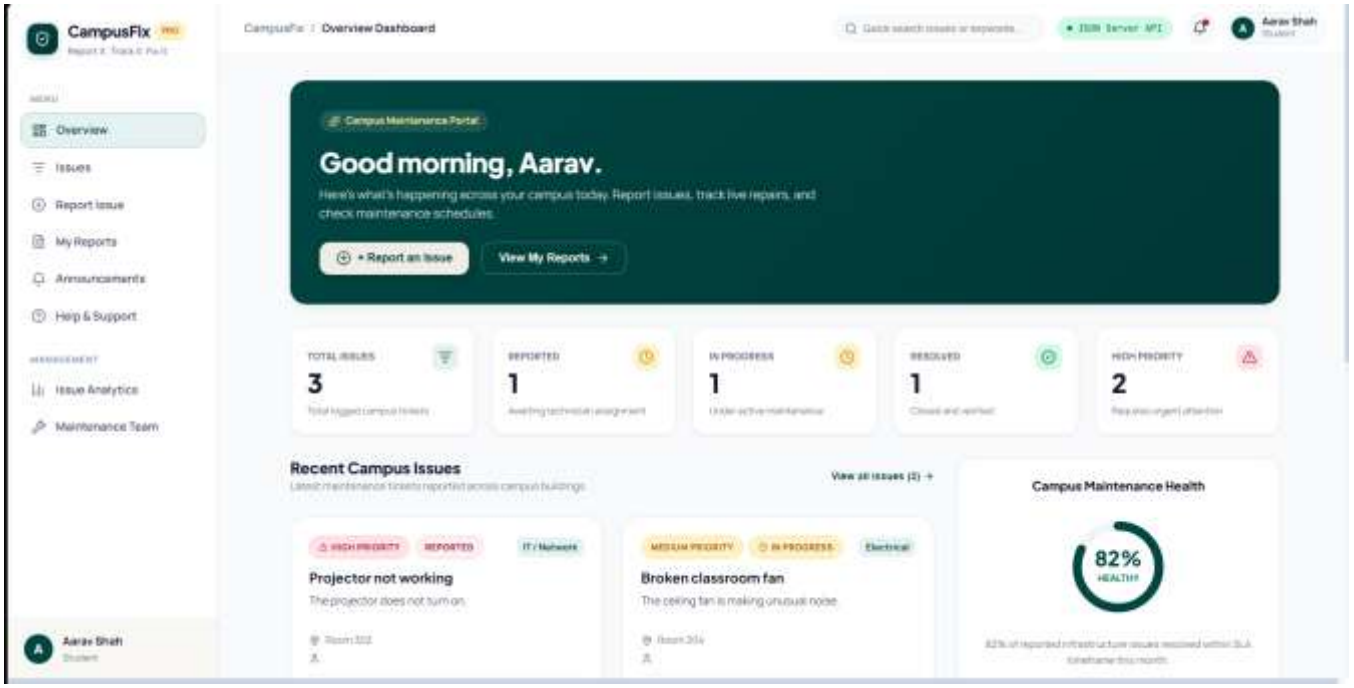
    // Actions & Setters
    loadIssues,
    addIssue,
    updateIssue: handleUpdateIssue,
    patchIssue: handlePatchIssue,
    deleteIssue: handleDeleteIssue,
    fetchSingleIssue,
    setSearchTerm,
    setSelectedCategory,
    setSelectedPriority,
    setSelectedStatus,
    setDataSource,
  };

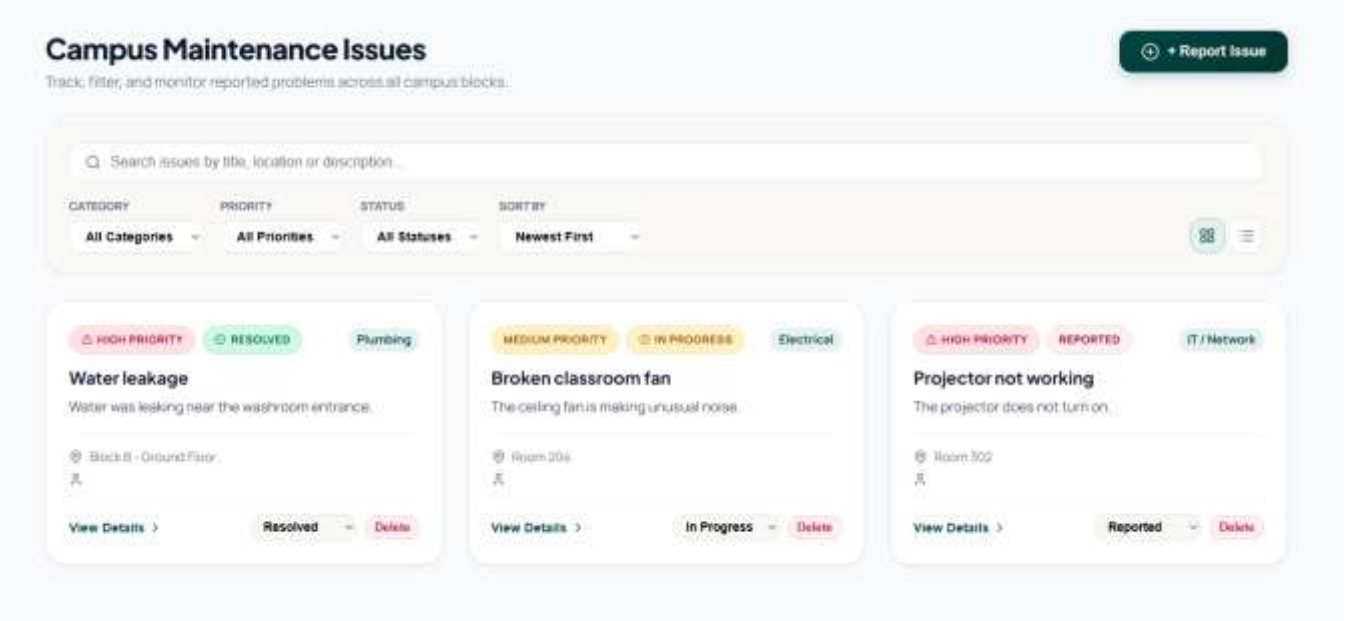
  return (
    <CampusFixContext.Provider value={value}>
      {children}
    </CampusFixContext.Provider>
  );
}
```

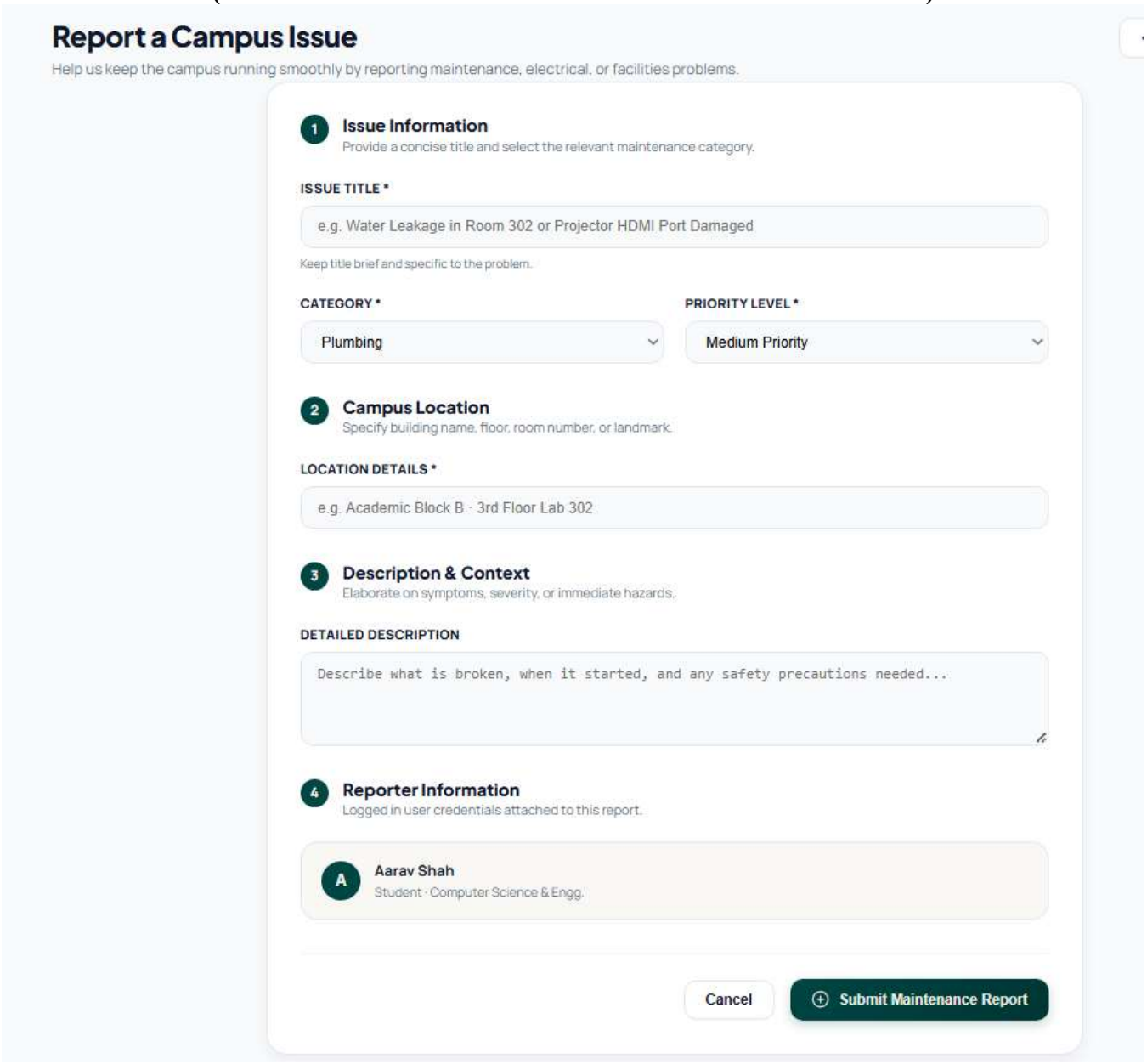
```
// Custom Hook to consume Context
export function useCampusFix() {
  const context = useContext(CampusFixContext);
  if (!context) {
    throw new Error('useCampusFix must be used within a CampusFixProvider');
  }
  return context;
}

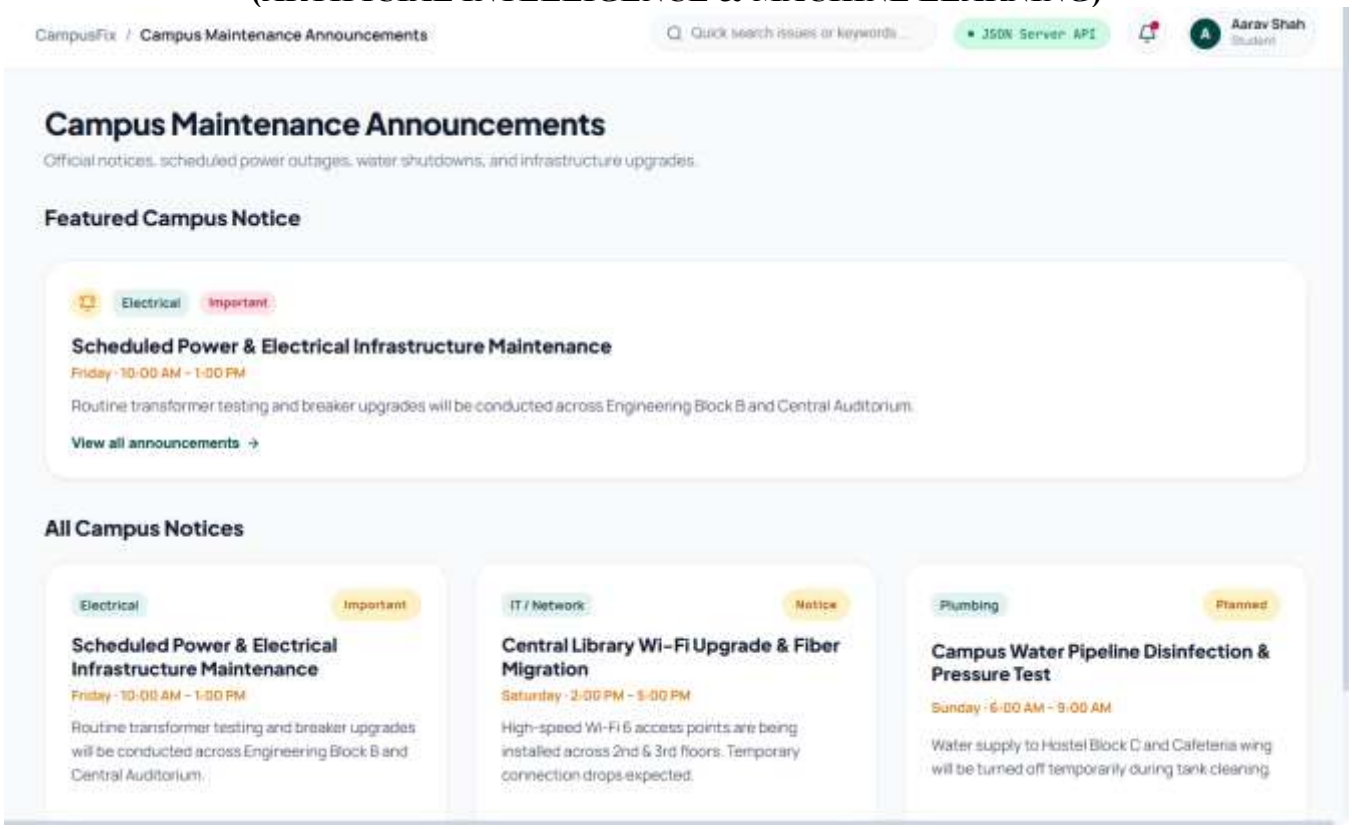
export default CampusFixContext;
```

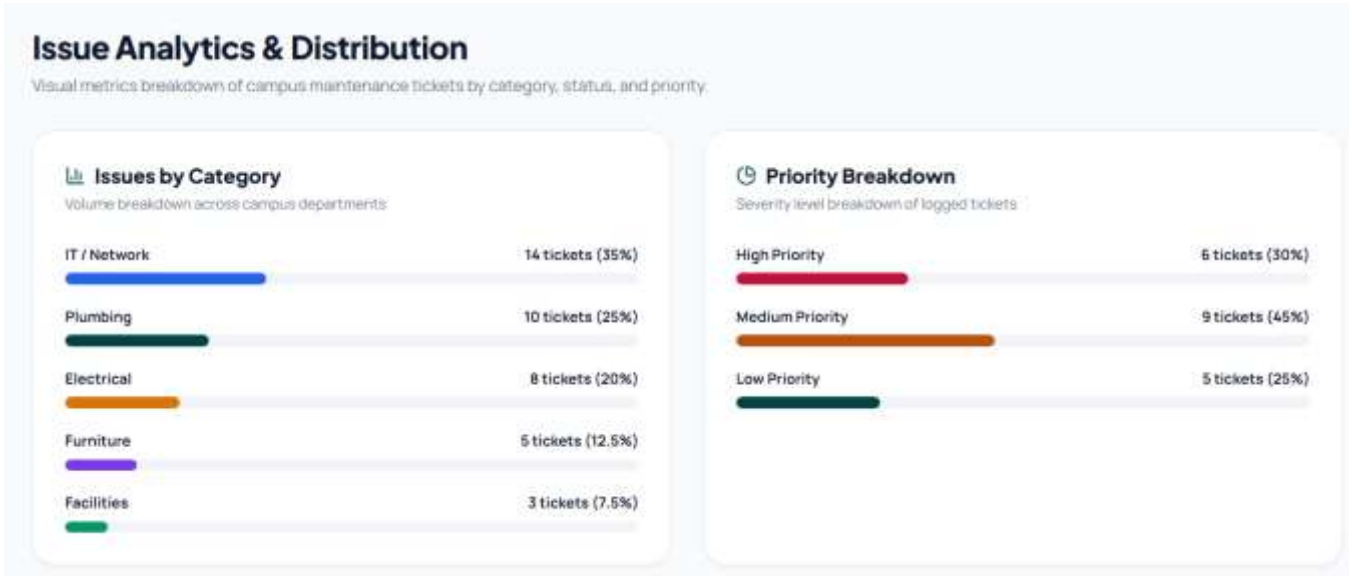
Output:




Campus Maintenance Issues
Track, filter, and monitor reported problems across all campus blocks.
+ Report Issue
Search issues by title, location or description
CATEGORY
All Categories
PRIORITY
All Priorities
STATUS
All Statuses
SORT BY
Newest First
HIGH PRIORITY
RESOLVED
Plumbing
Water leakage
Water was leaking near the washroom entrance.
Block B - Ground Floor
View Details
Resolved
Delete
MEDIUM PRIORITY
IN PROGRESS
Electrical
Broken classroom fan
The ceiling fan is making unusual noise.
Room 204
View Details
In Progress
Delete
HIGH PRIORITY
REPORTED
IT / Network
Projector not working
The projector does not turn on.
Room 302
View Details
Reported
Delete


Report a Campus Issue
Help us keep the campus running smoothly by reporting maintenance, electrical, or facilities problems.
1
Issue Information
Provide a concise title and select the relevant maintenance category.
ISSUE TITLE *
e.g. Water Leakage in Room 302 or Projector HDMI Port Damaged
Keep title brief and specific to the problem.
CATEGORY *
Plumbing
PRIORITY LEVEL *
Medium Priority
2
Campus Location
Specify building name, floor, room number, or landmark.
LOCATION DETAILS *
e.g. Academic Block B - 3rd Floor Lab 302
3
Description & Context
Elaborate on symptoms, severity, or immediate hazards.
DETAILED DESCRIPTION
Describe what is broken, when it started, and any safety precautions needed...
4
Reporter Information
Logged in user credentials attached to this report.
A
Aarav Shah
Student · Computer Science & Engg.
Cancel
Submit Maintenance Report





Conclusion: Thus, we have designed and implemented state management using Hooks and Context API; added form handling and validation.